

Aluno: Vinicius Ramon Ferreira - R.A: 24360978-2

Estação 1:

- De caracteres a lexemas e tokens

01_tokens.png

O que fazer

1. Abra o arquivo da Estação 1 e copie todo o conteúdo para o compilador.
2. Substitua ALUNO e RA pelos seus dados. Use somente letras e números dentro das aspas.
3. Execute o programa sem alterar a entrada inicial.
4. Leia no console as colunas LEXEMA e TOKEN. Localize um identificador, um operador, um número inteiro e o ponto e vírgula.
5. Troque somente o valor da constante ENTRADA por uma atribuição personalizada.

```
static final String ENTRADA = "total_mg = valor + 25;";  
// Troque mg por suas iniciais e 25 pelos dois últimos dígitos do RA.
```

6. Execute novamente e confira se o novo identificador e o novo número aparecem na lista.
7. Tire o print com a linha ENTRADA, sua identificação e o console visíveis.
8. Salve a imagem como 01_tokens.png.

Observe

Conclusão guiada: o texto `total_mg` é um lexema. IDENTIFICADOR é a categoria, ou token, atribuída a esse lexema.

- ☒ Encontrei pelo menos dois lexemas classificados como IDENTIFICADOR.
- ☒ Percebi que lexema e token não são a mesma coisa.

```
import java.util.HashMap;  
import java.util.Map;  
  
public class Main {  
    static final String ALUNO = "Vinicius Ramon Ferreira";  
    static final String RA = "243609782";  
  
    static final String ENTRADA = "total_vrf = valor + 82;";  
  
    private static final Map<Character, String> SIMBOLOS = new HashMap<>();  
  
    static {  
        SIMBOLOS.put('=', "ATRIBUICAO");  
        SIMBOLOS.put('+', "SOMA");  
        SIMBOLOS.put('-', "SUBTRACAO");  
        SIMBOLOS.put('*', "MULTIPLICACAO");  
        SIMBOLOS.put('/', "DIVISAO");  
        SIMBOLOS.put('.', "PONTO_E_VIRGULA");  
        SIMBOLOS.put('(', "ABRE_PARENTESES");  
        SIMBOLOS.put(')', "FECHA_PARENTESES");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);  
        System.out.println("ENTRADA: " + ENTRADA);  
        System.out.println();  
        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");  
        System.out.println("-----");  
  
        analisar(ENTRADA);  
    }  
  
    static void analisar(String fonte) {  
        int posicao = 0;  
  
        while (posicao < fonte.length()) {  
            char atual = fonte.charAt(posicao);
```

Estação 2:

- O que o analisador léxico reconhece

02_lexico.png

tokens essenciais devem permanecer equivalentes.

3. No teste C, localize `int` e `while`. Depois localize inteiro e `intValor`.
4. Substitua o valor de `IDENTIFICADOR_PERSONALIZADO` por um nome formado com suas iniciais.

```
static final String IDENTIFICADOR_PERSONALIZADO = "nota_mg";  
// Exemplo: nota_mg. Não use espaços nem acentos.
```

5. Execute novamente. Confira que `int` e `while` aparecem como `PALAVRA_RESERVADA` e que seu nome aparece como `IDENTIFICADOR`.
6. No teste D, localize a mensagem `ERRO_LEXICO` referente ao caractere `#`.
7. Tire um print mostrando seu identificador personalizado e a parte do console com `PALAVRA_RESERVADA`, `IDENTIFICADOR` e `ERRO_LEXICO`.
8. Salve a imagem como `02_lexico.png`.

Observe

- ☒ Espaços adicionais não criaram novos tokens.
- ☒ O comentário foi ignorado pelo analisador.
- ☒ `int` e `inteiro` foram classificados de maneira diferente.
- ☒ O caractere `#` foi rejeitado pelo analisador didático.

Ligação com os slides: o programa agrupa letras e dígitos em classes de caracteres e consulta uma tabela para decidir se um nome é uma palavra reservada.

The screenshot shows the OneCompiler IDE with a Java file named `Main.java`. The code defines a lexical analyzer that processes test cases. The console output shows the results of the analysis for three test cases:

```
=== TESTE C2 - identificador personalizado ===  
ENTRADA: int nota_vrf = 10;  
LEXEMA          CLASSE          TOKEN  
-----  
int              LETRA  
PALAVRA_RESERVADA  
nota_vrf         LETRA  
IDENTIFICADOR  
=                SIMBOLO  
ATRIBUICAO  
10              DIGITO  
INTEIRO  
;                SIMBOLO  
PONTO_E_VIRGULA  
  
=== TESTE D - caractere inválido ===  
ENTRADA: int valor# = 1;  
LEXEMA          CLASSE          TOKEN  
-----  
int              LETRA  
PALAVRA_RESERVADA  
valor            LETRA  
IDENTIFICADOR  
#                DESCONHECIDO  
ERRO_LEXICO  
=                SIMBOLO  
ATRIBUICAO  
1                DIGITO  
INTEIRO  
;                SIMBOLO  
PONTO_E_VIRGULA
```

Estação 3:

- Quando o compilador encontra erros

03_erro_compilador.png

Use os diagnósticos do compilador para reparar um programa Java curto.

Tempo	9 minutos
Arquivo	03-diagnostics/Main.java
Evidências	03_erro_compilador.png e 04_programa_corrigido.png

Importante: este arquivo foi entregue com erros propositalis. Nesta estação, falhar na primeira compilação significa que o experimento está funcionando.

Primeira compilação

1. Abra o arquivo, preencha ALUNO e RA e tente compilar sem corrigir as linhas marcadas.
2. Leia a primeira mensagem de erro. Procure o número da linha e algum símbolo esperado pelo compilador.
3. Sem corrigir ainda, tire um print com o código e a mensagem do compilador.
4. Salve a imagem como 03_erro_compilador.png.

Reparo mecânico

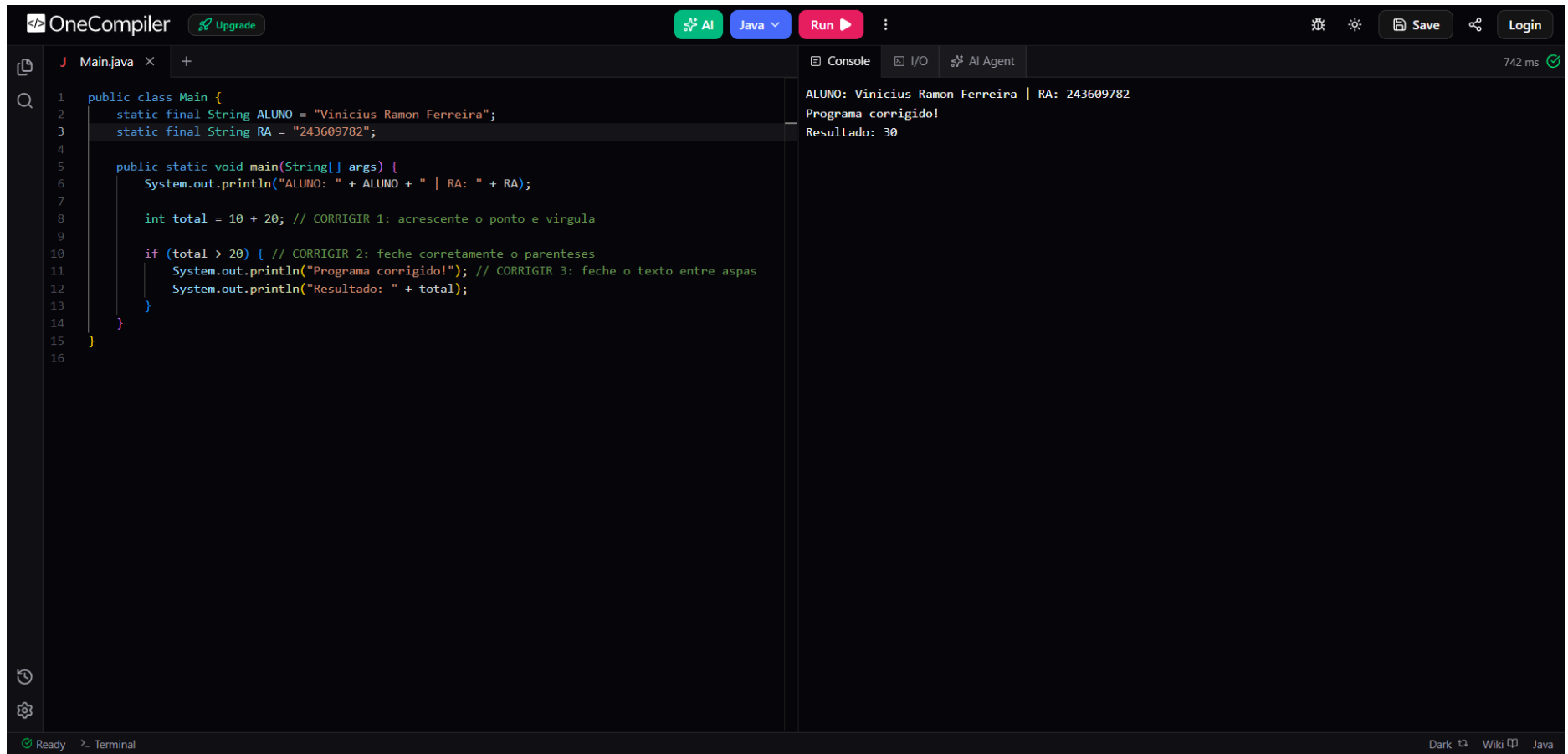
1. Procure as linhas marcadas com CORRIGIR 1, CORRIGIR 2 e CORRIGIR 3.
2. Acrescente o ponto e vírgula que está faltando.
3. Feche corretamente o parêntese da condição.
4. Feche corretamente o texto entre aspas.
5. Compile novamente. Se surgir outro erro, volte para a linha indicada e confira somente os sinais de pontuação.
6. Quando o programa executar, tire um novo print e salve como

```
1  ic class Main {
2  static final String ALUNO = "Vinicius Ramon Ferreira";
3  static final String RA = "243609782";
4
5  public static void main(String[] args) {
6      System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8      int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
9
10     if (total > 20 { // CORRIGIR 2: feche corretamente o pa
11         System.out.println("Programa corrigido!"); // CORRIG
12         System.out.println("Resultado: " + total);
13     }
14 }
15
16
```

Console:

```
Main.java:8: error: ';' expected
    int total = 10 + 20 // CORRIGIR 1: acrescente o
    ponto e vírgula
                        ^
Main.java:10: error: ')' expected
    if (total > 20 { // CORRIGIR 2: feche
    corretamente o parenteses
                        ^
Main.java:11: error: unclosed string literal
        System.out.println("Programa corrigido!"); //
CORRIGIR 3: feche o texto entre aspas
                        ^
3 errors
```

04_programa_corrigido.png



The image shows a screenshot of the OneCompiler web-based IDE. The interface includes a top bar with the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Java', and 'Run'. On the right side of the top bar, there are icons for settings, a 'Save' button, and a 'Login' button. Below the top bar, the editor area is divided into two main sections: a code editor on the left and a console on the right. The code editor shows a Java file named 'Main.java' with the following code:

```
1 public class Main {
2     static final String ALUNO = "Vinicius Ramon Ferreira";
3     static final String RA = "243609782";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16
```

The console on the right shows the output of the program:

```
ALUNO: Vinicius Ramon Ferreira | RA: 243609782
Programa corrigido!
Resultado: 30
```

The console also indicates a execution time of 742 ms. At the bottom of the IDE, there is a status bar showing 'Ready' and a 'Terminal' tab. The bottom right corner of the status bar includes links for 'Dark', 'Wiki', and 'Java'.

Estação 4:

- Parser descendente recursivo

05_descendente.png

Parser descendente recursivo

Acompanhe as chamadas de `expr`, `term` e `factor` enquanto o parser percorre a expressão.

Tempo	14 minutos
Arquivo	04-parser-descendente/Main.java
Evidência	05_descendente.png

Execução válida

- Abra o arquivo, preencha `ALUNO` e `RA` e execute a entrada original `2 + 3 * 4`.
- No console, acompanhe as mensagens `ENTRANDO` e `SAINDO` para `expr`, `term` e `factor`.
- Observe que a multiplicação é tratada dentro de `term` antes de `expr` terminar a soma.
- Troque a constante `ENTRADA` pela expressão personalizada abaixo.

```
static final String ENTRADA = "(5 + 3) * 4";  
// Troque 5 pelo último dígito não nulo do seu RA.
```

- Execute e procure a mensagem `ENTRADA ACEITA`.
- Tire um print mostrando a expressão, sua identificação e parte do rastreamento.
- Salve a imagem como `05_descendente.png`.

OneCompiler

Upgrade

AI

Java

Run

:

🔍

🌙

📄 Save

🔗

Login

Main.java

+

📄

🔍

1 public class Main {
2 static final String ALUNO = "Vinicius Ramon Ferreira";
3 static final String RA = "243609782";
4
5 static final String ENTRADA = "(5 + 3) * 2";
6 static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8 public static void main(String[] args) {
9 System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10 System.out.println("ENTRADA: " + ENTRADA);
11 System.out.println();
12
13 if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14 demonstrarRecursoaEsquerda(0);
15 System.out.println();
16 }
17
18 try {
19 Parser parser = new Parser(ENTRADA);
20 int resultado = parser.analisar();
21 System.out.println();
22 System.out.println("ENTRADA ACEITA");
23 System.out.println("RESULTADO DA EXPRESSAO: " + resulta
24 } catch (ErroSintatico erro) {
25 System.out.println();
26 System.out.println("ERRO SINTATICO: " + erro.getMessage
27 }
28
29
30 static void demonstrarRecursoaEsquerda(int nivel) {
31 System.out.println(recuo(nivel)
32 | | | + "expr -> expr + term (chamada " + (nivel + 1) +
33
34 if (nivel == 7) {
35 System.out.println(recuo(nivel + 1)
36 | | | + "DEMONSTRACAO INTERROMPIDA: a regra chamou ex
37 | | | + " sem consumir a entrada.");
38 }
39 }
40 }
41 }
42

Console

I/O

AI Agent

989 ms

ENTRANDO em <expr> | lookahead = (
ENTRANDO em <term> | lookahead = (
ENTRANDO em <factor> | lookahead = (
Abrindo parenteses
Novo lookahead: 5
ENTRANDO em <expr> | lookahead = 5
ENTRANDO em <term> | lookahead = 5
ENTRANDO em <factor> | lookahead = 5
Numero reconhecido: 5
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 2
ENTRANDO em <factor> | lookahead = 2
Numero reconhecido: 2
Novo lookahead: \$FIM\$
SAINDO de <factor> | lookahead = \$FIM\$
SAINDO de <term> | lookahead = \$FIM\$
SAINDO de <expr> | lookahead = \$FIM\$
ENTRADA ACEITA

Estação 5:

- Parser ascendente: deslocar e reduzir

06_ascendente.png

Tempo	9 minutos
Arquivo	05-parser-ascendente/Main.java
Evidência	06_ascendente.png

O que fazer

1. Abra o arquivo, preencha ALUNO e RA e execute a entrada `id+id*id`.
2. Leia as três colunas: PILHA, ENTRADA e ACAO.
3. Localize uma ação SHIFT. Nela, um item da entrada é deslocado para a pilha.
4. Localize uma ação REDUCE. Nela, uma sequência reconhecida é substituída pelo lado esquerdo de uma regra.
5. Localize a ação ACCEPT e tire um print com a identificação e as três colunas visíveis.
6. Salve a imagem como 06_ascendente.png.
7. Troque a entrada por `id+*id`, execute novamente e localize a ação ERROR. Não é necessário tirar outro print.

```
static final String ENTRADA_VALIDA = "id+id*id";
static final String ENTRADA_INVALIDA = "id+*id";
```

Compare as duas direções

- ☒ Descendente: parte de `expr` e avança em direção aos componentes menores.
- ☒ Ascendente: parte dos tokens e realiza reduções até chegar ao

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Main {
    static final String ALUNO = "Vinicius Ramon Ferreira";
    static final String RA = "243609782";

    static final String ENTRADA_VALIDA = "id+id*id";
    static final String ENTRADA_INVALIDA = "id+*id";

    // Na primeira execução, use ENTRADA_VALIDA. Depois, use ENTRADA_INVALIDA.
    static final String ENTRADA = ENTRADA_VALIDA;

    static final Map<Integer, Map<String, String>> ACTIONS = new HashMap<>();
    static final Map<Integer, Map<String, Integer>> GOALS = new HashMap<>();
    static final Map<Integer, Regra> REGRAS = new HashMap<>();

    static {
        // Gramática:
        // 1) E -> E + T    2) E -> T
        // 3) T -> T * F    4) T -> F
        // 5) F -> ( E )    6) F -> id

        regra(1, "E", 3, "E -> E + T");
        regra(2, "E", 1, "E -> T");
        regra(3, "T", 3, "T -> T * F");
        regra(4, "T", 1, "T -> F");
        regra(5, "F", 3, "F -> ( E )");
        regra(6, "F", 1, "F -> id");

        action(0, "id", "s5"); action(0, "(", "s4");
        action(1, "+", "s6"); action(1, "$", "acc");
        action(2, "+", "r2"); action(2, "*", "s7");
        action(2, ")", "r2"); action(2, "$", "r2");
        action(3, "+", "r4"); action(3, "*", "r4");
        action(3, ")", "r4"); action(3, "$", "r4");
    }

    public static void main(String[] args) {
        Console.println("ALUNO: " + ALUNO + " | RA: " + RA);
        Console.println("ENTRADA: " + ENTRADA);

        // Pilha, Entrada e Ação
        Stack<String> pilha = new Stack<>();
        String entrada = ENTRADA;
        String acao;

        while (true) {
            Console.println("PILHA: " + pilha + " | ENTRADA: " + entrada + " | AÇÃO: " + acao);

            if (acao == "SHIFT id -> estado 5") {
                pilha.push(entrada);
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE F -> id") {
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE T -> F") {
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE E -> T") {
                entrada = entrada.substring(1);
            } else if (acao == "SHIFT + -> estado 6") {
                pilha.push(entrada);
                entrada = entrada.substring(1);
            } else if (acao == "SHIFT id -> estado 5") {
                pilha.push(entrada);
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE F -> id") {
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE T -> F") {
                entrada = entrada.substring(1);
            } else if (acao == "SHIFT * -> estado 7") {
                pilha.push(entrada);
                entrada = entrada.substring(1);
            } else if (acao == "SHIFT id -> estado 5") {
                pilha.push(entrada);
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE F -> id") {
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE T -> T * F") {
                entrada = entrada.substring(1);
            } else if (acao == "REDUCE E -> E + T") {
                entrada = entrada.substring(1);
            } else if (acao == "ACCEPT") {
                Console.println("ACEPT");
                break;
            }
        }
    }
}
```